

R Lab: Week 7

Robert Gulotty

1/31/2017

```
library(httr)
library(dplyr, warn.conflicts = F)
library(purrr, warn.conflicts = F)
library(stringr)
```

This week, we are going to learn how to manipulate strings. Strings are a kind of datatype in R, made up of characters or letters. Strings are indicated by quotes on both sides. "For example"

Strings can be letters, numbers, or even spaces. We can make strings by using the `as.character()` command. `as.character(44)` `as.character(TRUE)`

Note that in R, strings can be in vectors or lists, and they can include NAs.

```
as.character(c(44, "banana", NA))
```

```
## [1] "44"      "banana" NA
```

The most useful package for string data is `stringr`. Here is an example: `str_replace`, which finds the pattern in the second entry, and replaces it with the third entry.

```
str_replace("banana", "n", "f")
```

```
## [1] "bafana"
```

```
str_replace_all("banana", "n", "f")
```

```
## [1] "bafafa"
```

We can also extract things from a vector of strings

```
str_extract_all("banana", "n")
```

```
## [[1]]
```

```
## [1] "n" "n"
```

```
str_extract_all(c("Alabama", "Alaska", "Arizona", "Arkansas"), "a")
```

```
## [[1]]
```

```
## [1] "a" "a" "a"
```

```
##
```

```
## [[2]]
```

```
## [1] "a" "a"
```

```
##
```

```
## [[3]]
```

```
## [1] "a"
```

```
##
```

```
## [[4]]
```

```
## [1] "a" "a"
```

Notice it returns a list, which we can turn into a simple vector with the `unlist` command.

So far, we have been looking for patterns in the data that are simple, single letters and the like. However, as you have already seen, there is a powerful programming language for pattern recognition in strings called regular expressions. This computer language is embedded in many other software packages, and is one of the most compact (and most difficult to read) programming languages. You can place regular expressions in many places that do patterns in R.

Regular Expressions

Square brackets

Regular expressions include being able to match multiple letters contained in `[]`. These can be interpreted asking for a single character that matches at one element of the set inside the square brackets.

```
str_replace("banana", "[bn]", "f")
```

```
## [1] "fanana"
```

```
str_replace_all("banana", "[bn]", "f")
```

```
## [1] "fafafa"
```

```
str_replace_all("banana cream pie", "[an]", "e")
```

```
## [1] "beeeeee cream pie"
```

We can get a range of letters using a hyphen. `[A-z]` represents all the capital letters and lower case letters.

```
str_replace_all("banana cream pie", "[a-z]", "e")
```

```
## [1] "eeeeeee eeeee eee"
```

Notice that regular expressions are case sensitive

```
str_replace_all("banana Cream pie", "[A-Z]", "e")
```

```
## [1] "banana eream pie"
```

Negative search

Regular expressions default to finding a match. We can also do negative search, with things in `[^ ]`. Here we find all the letters that are not `i`.

```
str_replace_all("Mississippi", "[^i]", "s")
```

```
## [1] "sissississi"
```

wildcards `\\d` `\\s` `\\w`

Regular expressions include wildcards for digits (`\\d`), spaces (`\\s`), and many characters (`\\w`) or any character (`.`)

```
str_replace("3 banana cream pies", "\\d", "4")
```

```
## [1] "4 banana cream pies"
```

```
str_replace("3 banana cream pies", "\\s", "-")
```

```
## [1] "3-banana cream pies"
```

```
str_replace("3 banana cream pies", "\\w", "9")
```

```
## [1] "9 banana cream pies"
```

```
str_replace("3 banana cream pies", ".", "2")
```

```
## [1] "2 banana cream pies"
```

```
str_replace("3 banana cream pies", "\\d\\s\\w", "B")
```

```
## [1] "Banana cream pies"
```

The opposite is done with capital letters

```
str_replace("3 banana cream pies", "\\D", "-")
```

```
## [1] "3-banana cream pies"
```

```
str_replace("3 banana cream pies", "\\S", "4")
```

```
## [1] "4 banana cream pies"
```

```
str_replace("3 banana cream pies", "\\W", "-")
```

```
## [1] "3-banana cream pies"
```

Repeated terms + * ?

Regular expressions become complicated with repeats.

+ matches 1 or more instances of a pattern that preceeds it.

```
str_replace("3 banana cream pies", "(an)+", "o")
```

```
## [1] "3 boa cream pies"
```

```
str_replace("3 bananana cream pies", "(an)+", "o")
```

```
## [1] "3 boa cream pies"
```

```
str_replace("3 bananana cream pies", "[A-z]+", "")
```

```
## [1] "3   cream pies"
```

* allows there to be 0 or more instances of a pattern that preceeds it.

```
str_replace("South Carolina", "South\\s*", "")
```

```
## [1] "Carolina"
```

```
str_replace("SouthCarolina", "South\\s*", "")
```

```
## [1] "Carolina"
```

```
str_replace("South-Carolina", "South\\s*", "")
```

```
## [1] "-Carolina"
```

Greedy vs Lazy evaluations

+ and * are 'greedy' meaning that they will search as far as they can:

```
str_extract("Alabama, Alaska, Arizona, Arkansas, California,", "A.+,")
```

```
## [1] "Alabama, Alaska, Arizona, Arkansas, California,"
```

If the command were lazy, it would stop after the first one it found. We can force these commands to stop at the first instance with a ? mark.

```
str_extract("Alabama, Alaska, Arizona, Arkansas, California", "A.+?,")
```

```
## [1] "Alabama,"
```

For if you actually want to match on question marks or any other wildcard, you need to put \\ in front.

```
str_replace("Who? are you?", "\\?", "")
```

```
## [1] "Who are you?"
```

Exercise: Pickle Industry Recipe

Here we get the recipes from the pickle packer association:

```
PickleRecipesSite <- GET("http://ilovepickles.org/book/export/html/2")
PickleRecipesText <- content(PickleRecipesSite, as = "text")
```

The recipes are one long character string.

We can use regular expressions to unpack these recipies.

First we will replace all the \n's, These are new line commands, and not useful for us.

```
PickleRecipesText <- str_replace_all(PickleRecipesText, "[\n]", " ")
```

Ok, Suppose I want a list of all the ingredients: We can do that by looking for everything between What you need, and the end of the section, marked by </div>. I noticed a pattern reading it. Now we will use extract, which returns a nested list. We can undo that with an unlist command.

```
ingredients <- unlist(str_extract_all(PickleRecipesText, "What You Need:&nbsp;</div>.+?</div>"))
ingredients <- str_replace_all(ingredients, "<.+?>", ", ")
```

to separate the ingredients, we “split” on commas, again, making a list.

```
ingredientslist <- str_split(ingredients, ",")
```

Remember, the map family of commands applies a function to each element of the list, and returns a list, Here we use str_trim.

```
trimingredients <- map(ingredientslist, str_trim)
```

Ok, now this next command is a little complicated. We calculate the list of each recipe with the map command, then we use that as the input into a rep command, which repeats each element of a vector a set number of times.

```
rep(c("a", "b"), c(1, 3))

## [1] "a" "b" "b" "b"
recipenumber <- rep(1:length(trimingredients), map(trimingredients, length))
```

Now we pass it do a data frame

```
dfpickles <- data.frame(recipenumber = recipenumber, ingredient = unlist(trimingredients))
```

Cut out some stuff that is not interesting

```
dfpickles <- dfpickles %>% filter(!ingredient %in% c("What You Need:&nbsp;", "&nbsp;",
  "")) %>% mutate(ingredient = tolower(ingredient), ingredient = str_replace(ingredient,
  "[\\d/-]+\\s\\w+", ""))
```